2. Funkcje. Zmienne globalne i lokalne

Lekcja porządkuje sposób dzielenia programu na podprogramy oraz pokazuje, jak zasięg i czas życia zmiennych wpływają na działanie kodu.

- funkcja jako nazwany podprogram w C++
- deklaracja, definicja i wywołanie funkcji
- parametry formalne i argumenty aktualne
- przekazywanie danych do funkcji i z funkcji
- zmienne globalne, lokalne i przesłanianie nazw

Po co tworzymy funkcje?

- dzielą złożony problem na mniejsze części
- pozwalają wielokrotnie użyć tego samego kodu
- ułatwiają testowanie, poprawianie i czytanie programu
- oddzielają szczegóły wykonania od głównego algorytmu

Dobra funkcja powinna wykonywać jedno jasno określone zadanie i mieć nazwę opisującą to zadanie.

Deklaracja funkcji

Deklaracja informuje kompilator o nazwie funkcji, typie zwracanej wartości oraz parametrach. Dzięki temu funkcja może zostać wywołana zanim kompilator napotka jej definicję.

```
typ_zwracany nazwa(lista_parametrow);

double poleProstokata(double bok1, double bok2);
void wypiszKomunikat();
```

Definicja funkcji

Definicja zawiera ciało funkcji, czyli blok instrukcji wykonywanych po jej wywołaniu.

```
double poleProstokata(double bok1, double bok2) {
    return bok1 * bok2;
}

void wypiszKomunikat() {
    cout << "Program dziala" << endl;
}
```

Funkcja zwracająca wartość i funkcja void

Rodzaj	Cechy	Przykład
zwracająca wartość	ma typ wyniku inny niż void; zwykle używa return	double srednia(...)
void	wykonuje działanie, ale nie zwraca wartości jako wyniku funkcji	void wyswietl(...)

Nie da się przeciążać funkcji wyłącznie typem zwracanej wartości - liczy się przede wszystkim lista parametrów.

Wywołanie funkcji

Wywołanie uruchamia kod zapisany w definicji funkcji. Argumenty aktualne powinny odpowiadać parametrom formalnym liczbą, kolejnością i typami.

```
double bok1 = 4.0;
double bok2 = 6.0;

double pole = poleProstokata(bok1, bok2);
wypiszKomunikat();
```

Parametry wejściowe przekazywane przez wartość

Przy przekazywaniu przez wartość funkcja otrzymuje kopię argumentu. Zmiana parametru wewnątrz funkcji nie zmienia zmiennej z programu wywołującego.

```
double kwadrat(double x) {
    x = x * x;
    return x;
}
// zmienna przekazana jako argument poza funkcją nie zostanie zmieniona przez przypisanie do x
```

Parametry przez referencję i wskaźnik

- referencja może służyć do zwracania wyniku przez parametr wyjściowy
- referencja do const pozwala uniknąć kopiowania i chroni argument przed zmianą
- wskaźnik można wykorzystać podobnie, ale wymaga operacji na adresie i dereferencji

```
void obwodProstokata(double a, double b, double &wynik) {
    wynik = 2 * a + 2 * b;
}

double poleProstokata(const double &a, const double &b) {
    return a * b;
}
```

Zmienne globalne

Zmienna globalna jest zadeklarowana poza funkcjami. Jej zakres zwykle obejmuje cały plik od miejsca deklaracji, a czas życia trwa przez cały czas działania programu.

Zmienne globalne mogą ułatwić prosty przykład, ale w większym programie często utrudniają kontrolę danych. Lepiej przekazywać dane przez parametry funkcji.

Zmienne lokalne

Zmienna lokalna jest zadeklarowana w funkcji albo w bloku kodu. Istnieje tylko w swoim zakresie widoczności.

```
int main() {
    double bok = 4;      // lokalna w main
    {
        double bok = 10; // lokalna tylko w bloku - przesłania poprzednią nazwę
    }
}
```

Przesłanianie nazw

Jeżeli w zakresie lokalnym pojawi się zmienna o tej samej nazwie co zmienna globalna, nazwa lokalna przesłania globalną.

```
double bok = 5; // globalna

int main() {
    double bok = 2; // lokalna przesłania globalną
    cout << bok;    // wypisze 2
}
```

Najważniejsze wnioski

- funkcja ma nagłówek i ciało; deklaracja nie jest tym samym co definicja
- parametry formalne należą do funkcji, a argumenty aktualne pojawiają się przy wywołaniu
- dane można przekazywać przez wartość, referencję lub wskaźnik
- zasięg zmiennej decyduje, gdzie zmienna jest widoczna
- czas życia zmiennej decyduje, jak długo istnieje w pamięci programu